JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.zip

Class InflaterInputStream

java.lang.Object
  java.io.InputStream
    java.io.FilterInputStream
      java.util.zip.InflaterInputStream

All Implemented Interfaces:

`Closeable`, `AutoCloseable`

Direct Known Subclasses:

`GZIPInputStream`, `ZipInputStream`

```
public class InflaterInputStream
extends FilterInputStream
```

This class implements a stream filter for uncompressing data in the "deflate" compression format. It is also used as the basis for other decompression filters, such as GZIPInputStream.

See Also:

`Inflater`

Field Summary

Fields

Modifier and Type	Field	Description
`protected byte[]`	**`buf`**	Input buffer for decompression.
`protected Inflater`	**`inf`**	Decompressor for this stream.

protected int	**len**	Length of input buffer.

Fields inherited from class java.io.FilterInputStream

in

Constructor Summary

Constructors

Constructor	Description
InflaterInputStream(**InputStream** in)	Creates a new input stream with a default decompressor and buffer size.
InflaterInputStream(**InputStream** in, **Inflater** inf)	Creates a new input stream with the specified decompressor and a default buffer size.
InflaterInputStream(**InputStream** in, **Inflater** inf, int size)	Creates a new input stream with the specified decompressor and buffer size.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
int	**available**()	Returns 0 after EOF has been reached, otherwise always return 1.

void	**close**()	Closes this input stream and releases any system resources associated with the stream.
protected void	**fill**()	Fills input buffer with more data to decompress.
void	**mark**(int readlimit)	Marks the current position in this input stream.
boolean	**markSupported**()	Tests if this input stream supports the `mark` and `reset` methods.
int	**read**()	Reads a byte of uncompressed data.
int	**read**(byte[] b, int off, int len)	Reads uncompressed data into an array of bytes.
void	**reset**()	Repositions this stream to the position at the time the `mark` method was last called on this input stream.
long	**skip**(long n)	Skips specified number of bytes of uncompressed data.

Methods inherited from class java.io.FilterInputStream

read

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Detail

inf

```
protected Inflater inf
```

Decompressor for this stream.

buf

```
protected byte[] buf
```

Input buffer for decompression.

len

```
protected int len
```

Length of input buffer.

Constructor Detail

InflaterInputStream

```
public InflaterInputStream(InputStream in,
                           Inflater inf,
                           int size)
```

Creates a new input stream with the specified decompressor and buffer size.

Parameters:

`in - the input stream`

`inf - the decompressor ("inflater")`

`size - the input buffer size`

Throws:

`IllegalArgumentException - if size <= 0`

InflaterInputStream

```
public InflaterInputStream(InputStream in,
                           Inflater inf)
```

Creates a new input stream with the specified decompressor and a default buffer size.

Parameters:

`in - the input stream`

`inf - the decompressor ("inflater")`

InflaterInputStream

```
public InflaterInputStream(InputStream in)
```

Creates a new input stream with a default decompressor and buffer size.

Parameters:

`in - the input stream`

Method Detail

read

```
public int read()
         throws IOException
```

Reads a byte of uncompressed data. This method will block until enough input is available for decompression.

Overrides:

`read` in class `FilterInputStream`

Returns:

`the byte read, or -1 if end of compressed input is reached`

Throws:

`IOException - if an I/O error has occurred`

See Also:

`FilterInputStream.in`

read

```
public int read(byte[] b,
                int off,
                int len)
         throws IOException
```

Reads uncompressed data into an array of bytes. If `len` is not zero, the method will block until some input can be decompressed; otherwise, no bytes are read and `0` is returned.

Overrides:

`read` in class `FilterInputStream`

Parameters:

`b - the buffer into which the data is read`

off - the start offset in the destination array b

len - the maximum number of bytes read

Returns:

the actual number of bytes read, or -1 if the end of the compressed input is reached or a preset dictionary is needed

Throws:

NullPointerException - If b is null.

IndexOutOfBoundsException - If off is negative, len is negative, or len is greater than b.length - off

ZipException - if a ZIP format error has occurred

IOException - if an I/O error has occurred

See Also:

FilterInputStream.in

available

```
public int available()
              throws IOException
```

Returns 0 after EOF has been reached, otherwise always return 1.

Programs should not count on this method to return the actual number of bytes that could be read without blocking.

Overrides:

available in class FilterInputStream

Returns:

1 before EOF and 0 after EOF.

Throws:

IOException - if an I/O error occurs.

skip

```
public long skip(long n)
          throws IOException
```

Skips specified number of bytes of uncompressed data.

Overrides:

skip in class FilterInputStream

Parameters:

n - the number of bytes to skip

Returns:

the actual number of bytes skipped.

Throws:

`IOException` - if an I/O error has occurred

`IllegalArgumentException` - if n < 0

close

```
public void close()
           throws IOException
```

Closes this input stream and releases any system resources associated with the stream.

Specified by:

`close` in interface `Closeable`

Specified by:

`close` in interface `AutoCloseable`

Overrides:

`close` in class `FilterInputStream`

Throws:

`IOException` - if an I/O error has occurred

See Also:

`FilterInputStream.in`

fill

```
protected void fill()
             throws IOException
```

Fills input buffer with more data to decompress.

Throws:

`IOException` - if an I/O error has occurred

markSupported

```
public boolean markSupported()
```

Tests if this input stream supports the `mark` and `reset` methods. The `markSupported` method of `InflaterInputStream` returns `false`.

Overrides:

markSupported in class FilterInputStream

Returns:

a boolean indicating if this stream type supports the mark and reset methods.

See Also:

InputStream.mark(int), InputStream.reset()

mark

```
public void mark(int readlimit)
```

Marks the current position in this input stream.

The mark method of InflaterInputStream does nothing.

Overrides:

mark in class FilterInputStream

Parameters:

readlimit - the maximum limit of bytes that can be read before the mark position becomes invalid.

See Also:

InputStream.reset()

reset

```
public void reset()
           throws IOException
```

Repositions this stream to the position at the time the mark method was last called on this input stream.

The method reset for class InflaterInputStream does nothing except throw an IOException.

Overrides:

reset in class FilterInputStream

Throws:

IOException - if this method is invoked.

See Also:

InputStream.mark(int), IOException

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.